

BẢN ĐỀ XUẤT DỰ ÁN

AWS IoT Monitoring and Control Dashboard

Hệ thống giám sát môi trường và điều khiển thiết bị từ xa sử dụng YOLO UNO, FastAPI, PostgreSQL, React và AWS.

Phiên bản tiếng Việt

Thực hiện cho chương trình First Cloud Journey / AWS Study Group

Ngày lập proposal: 15/06/2026

1. Tóm tắt dự án

AWS IoT Monitoring and Control Dashboard là một nền tảng IoT kết nối với cloud, được xây dựng để giám sát dữ liệu môi trường và điều khiển thiết bị từ xa trong một phòng.

Hệ thống sử dụng thiết bị **YOLO UNO ESP32-S3** để thu thập nhiệt độ, độ ẩm và cường độ ánh sáng. Phần cứng gửi telemetry thông qua HTTP REST API đến **FastAPI backend chạy trên Amazon EC2**. Backend lưu telemetry và command điều khiển vào **Amazon RDS for PostgreSQL**.

Dashboard được xây dựng bằng **React + Vite**, có chức năng hiển thị dữ liệu mới nhất, lịch sử cảm biến, trạng thái thiết bị và kết quả command. Người dùng có thể điều khiển quạt, đèn và rèm trên dashboard. Thiết bị định kỳ lấy command đang chờ từ backend, thực thi và gửi ACK để trạng thái command chuyển từ **Pending** sang **Executed**.

Amazon CloudWatch được sử dụng để thu thập log backend và metric hạ tầng. CloudWatch Alarms được cấu hình để theo dõi trạng thái vận hành của EC2 và RDS.

2. Bài toán cần giải quyết

2.1 Vấn đề hiện tại

Trong các phòng nhỏ, phòng thí nghiệm hoặc môi trường học tập, cảm biến và thiết bị điện thường hoạt động riêng lẻ. Điều này tạo ra một số hạn chế:

- Dữ liệu môi trường không được lưu tập trung.
- Người dùng không thể xem lại lịch sử nhiệt độ, độ ẩm và ánh sáng.
- Các thiết bị không thể được điều khiển từ xa trên một dashboard duy nhất.
- Một request điều khiển thành công chưa chứng minh thiết bị vật lý đã thực sự thực thi.
- Log ứng dụng và metric hạ tầng khó được theo dõi tập trung.

2.2 Giải pháp đề xuất

Project xây dựng một nền tảng giám sát và điều khiển trên cloud với luồng dữ liệu hai chiều hoàn chỉnh:

Hệ thống hỗ trợ:

- Nhận telemetry từ phần cứng thật.
- Hiển thị dữ liệu cảm biến mới nhất và dữ liệu lịch sử.
- Điều khiển quạt, đèn và rèm từ xa.
- Theo dõi command bằng trạng thái **Pending** và **Executed**.
- Nhận ACK từ thiết bị sau khi thực thi command.
- Phân tích dữ liệu và đưa ra đề xuất dựa trên rule.
- Giám sát bằng CloudWatch Logs, Metrics và Alarms.

2.3 Lợi ích

- Giám sát tập trung dữ liệu cảm biến và thiết bị chấp hành.
- Giao tiếp hai chiều giữa dashboard và phần cứng vật lý.
- Lưu bền vững telemetry và lịch sử command trong PostgreSQL.
- Hỗ trợ debug thông qua log, metric và trạng thái command.
- Thiết kế module hóa, có thể mở rộng cho nhiều phòng và thiết bị.

3. Mục tiêu và phạm vi dự án

3.1 Mục tiêu

Dự án hướng đến các mục tiêu sau:

1. Xây dựng thiết bị IoT vật lý bằng YOLO UNO ESP32-S3.
2. Thu thập nhiệt độ, độ ẩm và dữ liệu cảm biến ánh sáng.
3. Gửi telemetry đến FastAPI REST API trên Amazon EC2.
4. Lưu telemetry và command trong Amazon RDS for PostgreSQL.
5. Hiển thị dữ liệu mới nhất và lịch sử trên dashboard React.
6. Điều khiển quạt, đèn và rèm từ xa.
7. Triển khai vòng đời command **Pending** → **Executed**.
8. Gửi ACK từ thiết bị sau khi thực thi command.
9. Giám sát EC2, RDS và log backend bằng Amazon CloudWatch.
10. Hoàn thiện Workshop và báo cáo song ngữ.

3.2 Phạm vi hiện tại

Phiên bản hiện tại tập trung vào một thiết bị mẫu:

```
DEVICE_ID = room_01
```

Thiết bị gồm:

- Cảm biến nhiệt độ và độ ẩm DHT20.
- Cảm biến ánh sáng analog.
- Module quạt.
- Module đèn hoặc relay.
- Servo motor điều khiển rèm.

Các command được hỗ trợ:

FAN_ON
FAN_OFF
LIGHT_ON
LIGHT_OFF
CURTAIN_OPEN
CURTAIN_CLOSE

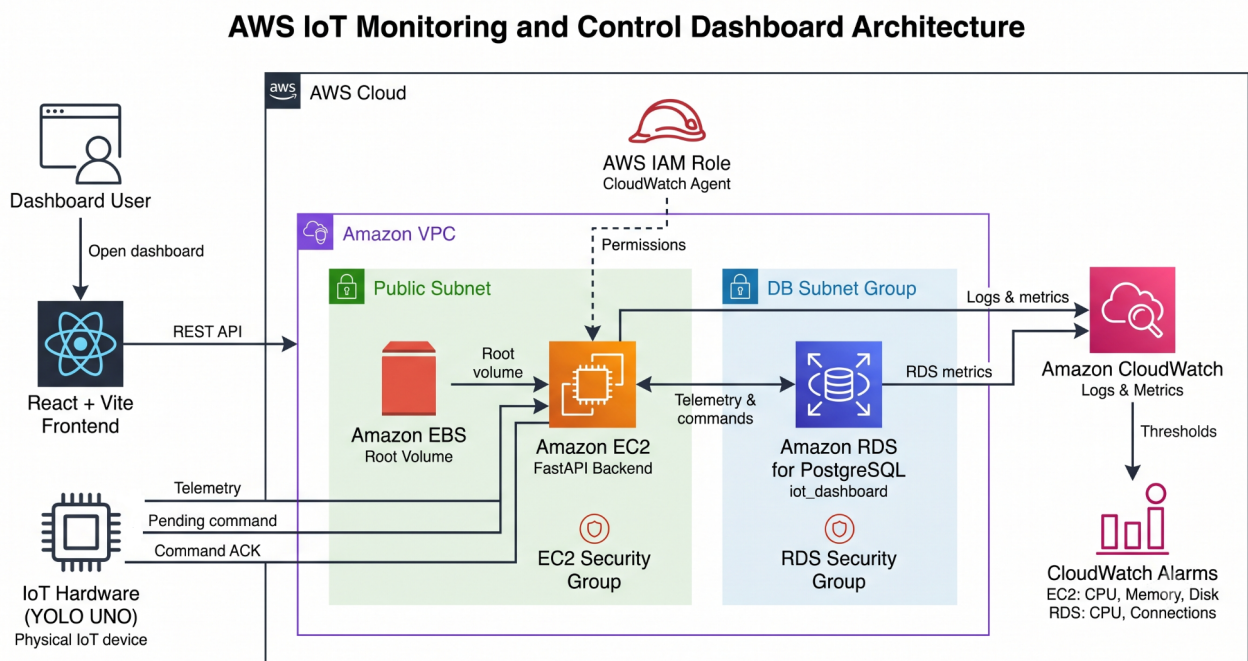
3.3 Ngoài phạm vi

Phiên bản hiện tại không sử dụng:

- AWS IoT Core.
- AWS Lambda.
- Amazon API Gateway.
- Amazon S3.
- Amazon SNS.
- Amazon ECS hoặc ECR.
- Amazon Cognito.
- Amazon CloudFront.
- Amazon DynamoDB.

Các dịch vụ này có thể được cân nhắc trong tương lai nếu project thay đổi mô hình triển khai.

4. Kiến trúc giải pháp



Hình 1. Kiến trúc AWS IoT Monitoring and Control Dashboard.

4.1 Các thành phần kiến trúc

Thành phần	Công nghệ	Mục đích
Frontend	React, Vite, TypeScript, Tailwind CSS	Hiển thị telemetry, lịch sử, phân tích và các nút điều khiển
Backend	Python, FastAPI, Uvicorn, SQLAlchemy, Pydantic	Cung cấp REST API và xử lý telemetry, command và ACK
Database	Amazon RDS for PostgreSQL	Lưu telemetry và trạng thái command
Compute	Amazon EC2 với Amazon EBS	Chạy FastAPI backend bằng systemd
IoT Hardware	YOLO UNO ESP32-S3, PlatformIO, Arduino	Đọc cảm biến, điều khiển actuator, lấy command và gửi ACK
Networking	Amazon VPC, subnet, Security Groups	Kiểm soát kết nối giữa người dùng, EC2 và RDS
Identity	AWS IAM Role	Cấp quyền để EC2 gửi dữ liệu lên CloudWatch
Monitoring	Amazon CloudWatch và CloudWatch Alarms	Thu thập log, metric và đánh giá các ngưỡng cảnh báo

4.2 Luồng dữ liệu

Luồng telemetry

- YOLO UNO đọc nhiệt độ, độ ẩm, ánh sáng và trạng thái thiết bị.
- Thiết bị gửi telemetry đến:

```
POST /api/telemetry
```

- FastAPI kiểm tra tính hợp lệ của payload.
- Backend lưu dữ liệu vào Amazon RDS for PostgreSQL.
- Frontend gọi backend để lấy dữ liệu mới nhất hoặc lịch sử.

Luồng command

- Người dùng nhấn nút điều khiển trên dashboard.
- Frontend gửi command đến:

```
POST /api/devices/{device_id}/commands
```

- Backend lưu command với trạng thái **Pending**.
- YOLO UNO định kỳ gọi:

```
GET /api/devices/{device_id}/commands/latest
```

1. Thiết bị thực thi command.
2. Thiết bị gửi ACK đến:

```
POST /api/devices/{device_id}/commands/{command_id}/ack
```

1. Backend cập nhật command thành **Executed**.

4.3 Lý do lựa chọn dịch vụ

Dịch vụ AWS	Lý do lựa chọn
Amazon EC2	Cho phép chủ động cấu hình FastAPI, Python environment, systemd và log file
Amazon EBS	Cung cấp root volume bền vững cho EC2
Amazon RDS for PostgreSQL	Database quan hệ được quản lý, phù hợp với telemetry và command
Amazon VPC	Cung cấp khả năng cô lập mạng và kiểm soát bằng Security Group
AWS IAM Role	Tránh hard-code AWS access key trên EC2
Amazon CloudWatch	Tập trung log, metric và alarm phục vụ vận hành và debug

4.4 Thiết kế bảo mật

- SSH chỉ cho phép từ IP quản trị.
- Port backend chỉ được mở theo nhu cầu của project demo.
- RDS chỉ nhận PostgreSQL traffic từ EC2 Security Group.
- Không hard-code AWS access key trong ứng dụng.
- EC2 sử dụng IAM Role để CloudWatch Agent hoạt động.
- `.env`, `.pem`, private key và `hardware/include/secrets.h` được loại khỏi Git.
- Tài liệu public sử dụng placeholder thay cho credential thật.

5. Kế hoạch triển khai kỹ thuật

Giai đoạn 1 — Nền tảng AWS

- Thiết kế kiến trúc AWS.
- Cấu hình VPC, subnet, Security Groups và IAM Role.
- Khởi tạo EC2.
- Tạo Amazon RDS for PostgreSQL.

- Kiểm tra kết nối từ EC2 đến RDS.

Giai đoạn 2 — Backend và Database

- Khởi tạo FastAPI backend.
- Định nghĩa Pydantic schema và SQLAlchemy model.
- Xây dựng API nhận telemetry.
- Xây dựng API latest và history.
- Xây dựng API tạo command, lấy command và gửi ACK.
- Chạy backend bằng `aws-iot-backend` systemd service.

Giai đoạn 3 — Hardware

- Cấu hình YOLO UNO trong PlatformIO.
- Kết nối DHT20, cảm biến ánh sáng, quạt, đèn và servo.
- Kết nối board với Wi-Fi.
- Gửi telemetry đến backend EC2.
- Lấy command đang chờ.
- Thực thi command.
- Gửi ACK và chống thực thi lặp command.

Giai đoạn 4 — Frontend và Tích hợp

- Xây dựng dashboard bằng React + Vite.
- Hiển thị telemetry mới nhất và dữ liệu lịch sử.
- Thêm điều khiển quạt, đèn và rèm.
- Thêm chế độ thủ công và chế độ tự động hoặc đề xuất dựa trên rule.
- Tích hợp frontend với backend trên EC2.
- Debug toàn bộ luồng hệ thống.

Giai đoạn 5 — Monitoring và Tài liệu

- Cài đặt và cấu hình CloudWatch Agent.
- Thu thập backend log, EC2 memory và disk.
- Theo dõi EC2 CPU và metric RDS.
- Tạo CloudWatch Alarms.
- Kiểm thử end-to-end.
- Quay video demo.
- Hoàn thiện báo cáo và Workshop song ngữ.

6. Timeline và các mốc triển khai

Dự án được thực hiện trong **10 tuần**.

Tuần	Mốc triển khai	Kết quả mong đợi
Tuần 1	Phân tích yêu cầu và lập kế hoạch	Bài toán, phạm vi, phân công và kiến trúc ban đầu
Tuần 2	Thiết kế kiến trúc AWS và nền tảng mạng	VPC, subnet, Security Groups và kế hoạch IAM
Tuần 3	Triển khai EC2 và RDS	EC2 hoạt động và PostgreSQL database sẵn sàng
Tuần 4	Xây dựng nền tảng backend và database	Cấu trúc FastAPI, schema và kết nối database
Tuần 5	Xây dựng API telemetry và command	Telemetry, latest, history, command và ACK endpoint
Tuần 6	Tích hợp phần cứng YOLO UNO	Đọc cảm biến, điều khiển actuator, Wi-Fi và REST API
Tuần 7	Phát triển frontend dashboard	Telemetry card, biểu đồ, bảng điều khiển và phân tích
Tuần 8	Tích hợp end-to-end và debug	Hoàn thiện luồng telemetry và command với phần cứng thật
Tuần 9	CloudWatch và kiểm thử hệ thống	Logs, metrics, alarms, failure test và security review
Tuần 10	Hoàn thiện tài liệu, demo và bàn giao	Báo cáo song ngữ, Workshop, video demo và repository cuối

7. Ước tính ngân sách

Project sử dụng các tài nguyên AWS nhỏ, phù hợp với mục đích học tập và demo.

Tài nguyên	Yếu tố tính phí	Cách tối ưu
Amazon EC2	Thời gian instance hoạt động	Dùng instance nhỏ và dừng khi không cần thiết
Amazon EBS	Dung lượng được cấp phát	Chỉ cấp root volume đủ dùng
Amazon RDS for PostgreSQL	Instance class, storage và thời gian chạy	Dùng database instance nhỏ cho môi trường Workshop
Amazon CloudWatch	Log ingestion, retention và custom metrics	Giới hạn thời gian lưu log và tránh metric không cần thiết
Data Transfer	Request giữa người dùng, hardware và EC2	Sử dụng chu kỳ telemetry và polling hợp lý

Chi phí thực tế phụ thuộc vào region, kích thước tài nguyên và thời gian sử dụng. Nhóm cần kiểm tra và clean-up tài nguyên sau Workshop để tránh phát sinh chi phí ngoài dự kiến.

8. Đánh giá rủi ro

Rủi ro	Ảnh hưởng	Phương án xử lý
YOLO UNO mất Wi-Fi	Telemetry và command tạm thời ngừng hoạt động	Tự kết nối lại Wi-Fi và retry HTTP request
Public IP của EC2 thay đổi	Frontend và hardware không gọi được backend	Cập nhật cấu hình hoặc dùng Elastic IP trong phiên bản tương lai
Backend bị dừng	API không hoạt động	Chạy Uvicorn bằng <code>systemd</code> và tự động restart
Không kết nối được RDS	Không lưu được telemetry và command	Kiểm tra endpoint, credential, Security Groups và <code>DATABASE_URL</code>
Command bị thực thi lặp	Actuator thực hiện lại hành động	Lưu command ID mới nhất và chỉ thực thi mỗi command một lần
Gửi ACK thất bại	Command vẫn ở trạng thái <code>Pending</code>	Retry ACK nhưng không thực thi lại actuator
Lộ credential	Rủi ro bảo mật	Ignore <code>.env</code> , <code>.pem</code> , private key và <code>secrets.h</code>
Phát sinh chi phí AWS	Chi phí project tăng	Dừng hoặc xóa tài nguyên sau khi kiểm thử và kiểm tra CloudWatch
Giá trị cảm biến không chính xác	Đề xuất điều khiển không phù hợp	Kiểm tra dữ liệu và hiệu chỉnh threshold
Không đồng nhất khi tích hợp	Frontend, backend và hardware dùng payload hoặc endpoint khác nhau	Duy trì một API contract chung và kiểm thử end-to-end

9. Kết quả kỳ vọng

Project dự kiến tạo ra:

- Một thiết bị IoT vật lý YOLO UNO hoạt động ổn định.
- Telemetry nhiệt độ, độ ẩm và ánh sáng được gửi định kỳ.
- FastAPI backend chạy trên Amazon EC2.
- Telemetry và command được lưu bền vững trong Amazon RDS for PostgreSQL.
- Dashboard React hiển thị dữ liệu mới nhất, lịch sử, điều khiển và đề xuất.

- Điều khiển quạt, đèn và rèm từ xa.
- Theo dõi command từ **Pending** sang **Executed**.
- ACK từ thiết bị sau khi hoàn thành command.
- CloudWatch logs, metrics và alarms.
- Workshop và báo cáo FCAJ song ngữ.
- Video demo thể hiện dashboard, database, CloudWatch và phần cứng thật.

Tiêu chí thành công

Project được xem là hoàn thành khi:

1. YOLO UNO gửi được telemetry hợp lệ đến backend.
2. Telemetry được lưu và đọc lại từ PostgreSQL.
3. Dashboard hiển thị được dữ liệu mới nhất và lịch sử.
4. Dashboard tạo được command cho **room_01**.
5. Hardware nhận và thực thi các command được hỗ trợ.
6. ACK cập nhật command thành **Executed**.
7. CloudWatch nhận được log và metric đã cấu hình.
8. Người đọc khác có thể làm theo Workshop để tái hiện các bước triển khai chính.

10. Phân công thành viên

Thành viên	Vai trò	Trách nhiệm chính
Phạm Lê Minh Khôi	AWS và Hardware Lead	Kiến trúc AWS, VPC, Security Groups, IAM Role, EC2, RDS, CloudWatch, DevOps, firmware YOLO UNO, cảm biến, actuator, telemetry, polling command và ACK
Ngô Minh Thuận	Backend Developer	FastAPI backend, API endpoint, Pydantic schema, SQLAlchemy model, tích hợp PostgreSQL, xử lý telemetry, vòng đời command và ACK
Thượng Đình Hưng	Frontend và Integration Developer	React + Vite dashboard, giao diện, trực quan hóa telemetry, điều khiển thiết bị, tích hợp tổng thể, debug và quay video demo
Lê Bảo Khánh	Documentation và QA	Proposal, blog, worklog theo tuần, báo cáo event, tài liệu Workshop, kiểm tra song ngữ, navigation, screenshot và đảm bảo chất lượng

11. Sản phẩm bàn giao

Các sản phẩm cuối cùng gồm:

- Source code repository.
- FastAPI backend.
- React + Vite frontend.
- Firmware PlatformIO cho YOLO UNO.
- Sơ đồ kiến trúc AWS.
- Database schema trên Amazon RDS.
- CloudWatch logs, metrics và alarms.
- Tài liệu API.
- Proposal và Workshop song ngữ.
- Worklog, blog và báo cáo event.
- Video demo end-to-end.
- Hướng dẫn clean-up và bàn giao project.